LITERATURE REVIEW DOCUMENT

1. Autonomous Multi-Agent System for Integrated SRE and Self-Healing in Cloud-Native Environments

Author(s): Manohar NV; Ravi Shukla; Dr. Shinu Abi

DOI / Identifier: 10.2991/978-94-6463-950-6_35

Date: 2025

Review / Notes:

Manohar, Shukla, and Abi (2025) present an autonomous self-healing framework for Site Reliability Engineering that uses a multi-agent architecture to automate the incident management lifecycle in cloud-native environments. The system is designed to operate across detection, triage, root cause analysis, remediation, and resolution rather than limiting automation to alert generation. The authors implement the framework using CrewAI and Large Language Models including DeepSeek-R1, GPT-4o, and Gemini 2.5 Flash.

The framework is organized into several layers. A Kubernetes infrastructure layer hosts both the target application and the autonomous agent system. An application layer contains the monitored application and a monitoring agent integrated with Middleware.io, which continuously observes application performance and system health. When an anomaly is detected, an alert starts the self-healing workflow. The central Autonomous SRE Agent layer contains the specialized agents that collaborate on the incident, while an external dependency layer connects the system with services such as Middleware.io, Jira Service Management, GitHub, and Langfuse.

The multi-agent system contains four main specialized roles. The Incident Resolution Manager acts as the orchestrator and manages the incident lifecycle, receives alerts, prioritizes incidents, and delegates tasks to other agents. The JIRA Manager handles incident-ticket operations, including creating tickets, updating findings, changing ticket states, and maintaining the incident workflow. The Root Cause Analyzer performs the technical investigation by correlating application logs from Middleware.io, Kubernetes cluster state, and relevant source code from GitHub. The Code Fix Generator takes the RCA output and generates the required code or configuration changes, creates a new Git branch, and submits a pull request for review.

The paper implements the complete workflow as a sequential incident-resolution process. An incident begins when a Jira ticket is created manually or through a monitoring alert. The Incident Resolution Manager selects high-priority incidents and delegates investigation to the appropriate agents. The Root Cause Analyzer collects information from the available operational sources and produces an RCA report, which is added to the Jira ticket. After RCA is completed, the Code Fix Generator uses the report to implement the required remediation and creates a GitHub pull request. A human engineer then reviews the generated pull request before it can be merged. If the incident is too complex for autonomous resolution, it is returned for manual intervention. After approval and merging, the incident ticket is transitioned to completion.

The implementation was developed using Python 3.12 and CrewAI. The agents were defined through prompt engineering that specified their roles, goals, and workflows. The Root Cause Analyzer was instructed to correlate Jira ticket information, Kubernetes state, application logs, and source code to identify the relevant code or configuration responsible for the problem. The Code Fix Generator was instructed to implement the remediation plan and generate a pull request through the GitHub workflow. Custom tools based on REST APIs were developed for Jira, Kubernetes, and GitHub interactions, including a Jira MCP Tool, Kubernetes analysis tools, and GitHub MCP Tools. The complete application was containerized and deployed on Kubernetes.

An important part of the implementation is its emphasis on auditability. Langfuse is integrated to record agent interactions and LLM calls throughout the workflow. This provides visibility into the decisions and actions taken by the multi-agent system and allows the incident-resolution process to be inspected after execution. The human-in-the-loop stage provides an additional control mechanism because generated code changes are reviewed before being merged into the main branch.

The authors evaluate the framework in a controlled environment designed to resemble a production SRE setup. A faulty web application is deployed on Kubernetes and engineered to generate operational failures such as internal server errors, database connection timeouts, and pod crashes. The environment is integrated with Middleware.io, Jira Service Management, GitHub, and Langfuse so that the complete incident lifecycle can be evaluated. Gemini 2.5 Pro is used as an independent judge to assess the quality and completeness of tasks performed by the agents.

The evaluation reports 71% correctness in agent task delegation, 68% precision for identifying actionable root causes, and up to 96% accuracy for simpler RCA tasks. The Code Fix Generator successfully produced valid pull requests for 73% of the tested cases. The JIRA Manager achieved 97% compliance with the expected ticket-state workflow, while the average end-to-end execution time was 27.48 minutes. Langfuse recorded 100% of the workflow executions, providing complete telemetry of the agent interactions.

The results show that the Root Cause Analyzer can correlate information from different sources to produce incident analyses, although performance decreases for more complex, multi-faceted problems. The Code Fix Generator demonstrates that the framework can move from diagnosis to an actual software change rather than stopping at an RCA report. The authors therefore demonstrate an end-to-end approach in which LLM agents can coordinate operational investigation and software remediation while retaining human approval for the final change.

The paper also identifies future extensions to the framework. The authors propose a long-term memory graph so that agents can learn from historical incidents and reuse proven resolution strategies, together with a searchable knowledge base of previous problems. They also specifically identify deeper integration with CI/CD pipelines and correlation of incidents with recent code changes as a way to further improve RCA precision.

For the proposed Autonomous CI/CD Incident Triager & Self-Healer, this paper is particularly relevant because it demonstrates the practical combination of multi-agent orchestration, cloud monitoring, Kubernetes analysis, source-code inspection, automated remediation, pull-request generation, human approval, and auditability. The proposed project can build on this type of end-to-end workflow while focusing more explicitly on the correlation of CI/CD events, Git changes, deployments, and production telemetry during incident investigation.

2. Exploring LLM-based Agents for Root Cause Analysis

Author(s): Devjeet Roy; Xuchao Zhang; Rashi Bhave; Chetan Bansal; Pedro Las-Casas; Rodrigo Fonseca; Saravan Rajmohan

DOI / Identifier: arXiv:2403.04123v1

Date: 7 March 2024

Review / Notes:

Roy et al. (2024) investigate the use of Large Language Model agents for automated Root Cause Analysis of cloud incidents. The paper focuses on the ReAct agent framework, in which an LLM alternates between reasoning and tool-use actions to investigate an incident. The central idea is that an RCA agent should not be limited to the information already contained in an incident report; it should be able to retrieve additional information and interact with diagnostic tools in a manner similar to an on-call engineer.

The authors build their RCA agent around the ReAct framework. In each iteration, the agent produces a reasoning step that determines the next action, selects a tool from a fixed action space, observes the tool output, and then continues reasoning. This process is repeated until the agent reaches a root-cause conclusion or the maximum number of iterations is reached. The implementation uses LangChain, and the authors limit the ReAct loop to a maximum of 20 iterations because of time and resource constraints.

The first evaluation examines the agent in a generalized setting without access to team-specific diagnostic services. The available tools include an incident-details tool that allows the agent to query information from the raw incident description and a historical-incidents tool that retrieves relevant past incidents. This setting is intended to establish how well an LLM agent can perform RCA using general-purpose information sources without requiring fine-tuning or specialized infrastructure integrations.

The historical-incident retrieval system uses two retrieval approaches. A dense retriever is implemented using the SentenceTransformers all-mpnet-base-v2 model with Max Marginal Relevance, while a sparse retriever uses BM25. The agent can generate retrieval queries during its reasoning process. One retrieval variant directly returns retrieved historical incidents to the agent, while another uses a two-step process in which the agent generates a query and then performs question answering over the retrieved incidents. The two-step approach helps separate the retrieval query from the target incident and reduces context-length problems.

The authors use a large internal incident dataset collected from an enterprise incident portal between January 2020 and September 2021. The collection contains approximately 107,000 unique incidents, divided into training, evaluation, and test sets. For the experiments, 100 evaluation incidents and 500 test incidents are sampled. Historical incidents from the training set form the retrieval corpus, while incident titles and descriptions provide the primary information about the target incidents.

GPT-4 with an 8,000-token context window is used as the main language model for the experiments, while GPT-3.5-Turbo is used for summarizing incident descriptions, root causes, and discussion comments to reduce context size and cost. The authors compare the ReAct agent with several non-agent baselines, including a Retrieval Baseline, zero-shot Chain-of-Thought, and Interleaving Retrieval with Chain-of-Thought. These baselines provide a comparison between retrieval, explicit reasoning, and agent-based reasoning.

The evaluation considers both automatic and qualitative measures. Automatic evaluation uses BLEU, METEOR, ROUGE-L, and BERTScore to compare generated root causes with reference answers. Because these metrics do not fully capture factual correctness, the authors also manually annotate predictions using categories such as precise, imprecise, hallucinated, insufficient evidence, reasoning error, and

retrieval error. This qualitative evaluation is important because an RCA system can generate a response that appears semantically similar to the reference while containing unsupported or misleading details.

In the generalized evaluation, the retrieval baseline improves on several lexical metrics as more historical incidents are retrieved, while semantic similarity changes only slightly. The ReAct variants retrieve approximately four unique historical incidents per incident on average, with about two retrieval lookups. The qualitative analysis shows that the ReAct approach has lower hallucination rates than the retrieval-only baseline and can sometimes filter out historical incidents that are lexically similar but diagnostically different. However, the ReAct agent also produces cases where it cannot determine a root cause because the available incident information is insufficient.

The authors also investigate whether discussion comments attached to historical incidents improve RCA. These comments contain records of diagnostic steps taken by on-call engineers. The experiments show mixed changes in lexical metrics across models, while semantic metrics indicate that adding discussion comments does not significantly change the resulting root-cause predictions. The authors explain that discussion comments often contain administrative information or diagnostic steps that depend on tools unavailable to the agent, limiting their usefulness when the agent cannot reproduce the same investigation process.

The second major part of the paper is a practical case study with the Azure Fundamental Team. Instead of using only generalized retrieval tools, the researchers adapt the ReAct agent to interact with team-specific diagnostic services. They work with the team over several weeks to understand its RCA process, identify the diagnostic services used by engineers, and develop tools that allow the agent to interact with those services.

The case-study agent is given access to several specialized tools. A Database Query Tool allows it to construct queries, execute them against the team's database system, and analyze the returned results. Query results are converted into Pandas DataFrames, after which a separate LLM-based DataFrame Query Engine can answer natural-language questions about the returned data. This allows the agent to perform both database retrieval and subsequent interpretation of structured results.

The agent also receives a KBA Question-Answering Tool for retrieving information from the team's knowledge-base articles. These articles contain troubleshooting procedures, database queries, system information, and other operational knowledge used by on-call engineers. The system uses a vector store over KBA chunks and an LLM to answer questions based on retrieved information. A separate KBA Planning Tool is introduced because the researchers observed that unrestricted ReAct reasoning could sometimes begin diagnostic actions without first forming an adequate high-level plan.

The case study also includes a Human Interaction Tool. The authors found that some incidents require information that the agent cannot directly obtain, such as manually reproduced errors or information available only to an engineer. The human interaction mechanism therefore allows the agent to request additional information from an on-call engineer. The interface also allows engineers to interrupt agent actions, verify tool executions, and provide explicit feedback.

One of the practical examples demonstrates why access to diagnostic services is important. For a cluster setting-drift incident, the incident report alone is insufficient to determine whether the incident is a false positive or requires remediation. The agent needs to query a database to identify affected clusters and analyze the results. It also needs information from the relevant KBA to understand the troubleshooting procedure and database details. This case illustrates how the agent can reproduce parts of the investigation workflow normally performed by an experienced engineer.

Overall, the paper demonstrates that an LLM agent can be used as an active participant in RCA rather than merely as a system that summarizes an incident report. The ReAct architecture allows the model to

decide what information to retrieve, observe the results, and use those observations to determine subsequent actions. The generalized experiments establish the capabilities and limitations of the approach, while the Azure Fundamental Team case study shows how specialized diagnostic tools and human interaction can make the approach more practical.

For the proposed Autonomous CI/CD Incident Triager & Self-Healer, this paper is particularly relevant to the Evidence Collection and RCA Engine because it demonstrates dynamic tool-based investigation. The proposed system can apply the same principle to logs, metrics, Git history, CI/CD results, deployment records, configuration information, and service health, allowing the agent to gather additional evidence as needed instead of relying only on the initial alert.

3. LogRCA: Log-based Root Cause Analysis for Distributed Services

Author(s): Thorsten Wittkopp; Philipp Wiesner; Odej Kao

DOI / Identifier: arXiv:2405.13599v1 (the source does not state a DOI)

Date: 22 May 2024

Review / Notes:

LogRCA focuses on root cause analysis in distributed services using log data as the main source of diagnostic evidence. Its relevance to the proposed Autonomous CI/CD Incident Triager & Self-Healer comes from addressing one of the most important problems in automated incident investigation: operational logs contain large amounts of information, but only a small portion may be useful for identifying the underlying cause of a failure. The work therefore supports the project's need for an evidence-selection and evidence-analysis stage rather than simply passing all available logs to an AI model.

For the proposed system, logs will be one of several evidence sources used by the incident investigation agent. When an alert is received, the agent should be able to retrieve logs related to the affected service and incident time window, identify the relevant error patterns and connect them with other operational information. LogRCA is therefore particularly relevant to the log-analysis component of the proposed architecture and to the broader objective of reducing the amount of irrelevant information presented to the reasoning layer.

The paper's distributed-service focus also aligns with the proposed project. In a cloud-native or microservice environment, a single incident can generate logs across several services and components. The visible error may occur in a downstream service even though the initiating problem originated elsewhere. This makes it necessary to consider the relationship between log events and the overall incident timeline rather than treating an individual error message as the root cause.

For our project, the main lesson is that log analysis should be integrated with other evidence rather than used as an isolated RCA mechanism. The proposed agent can use logs to identify symptoms and candidate causes, and then use Git history, CI/CD pipeline results, deployment history, metrics and service health to validate those candidates. For example, a database connection error in the logs could be investigated further by checking whether the database is actually unhealthy, whether a recent deployment changed database configuration, and whether the problem started immediately after that deployment.

This supports the proposed evidence-driven investigation workflow. Instead of asking the LLM to directly infer a root cause from raw logs, the system can first retrieve the relevant log window, identify important events, construct a timeline and then combine those events with evidence from the software-delivery and runtime environment. The resulting incident context can then be passed to the root-cause reasoning component.

The paper is therefore relevant mainly to the proposed project's observability and RCA stages. It provides a basis for treating logs as structured evidence that must be filtered and interpreted before being used for root-cause reasoning. In the overall project, this log-analysis capability becomes one tool available to the incident-investigation agent, alongside Git, CI/CD, deployment, metrics and service-health tools.

The proposed project extends this direction by placing log analysis inside a larger incident-response workflow. After evidence is collected and a root cause is identified, the system can generate a controlled remediation plan, execute an allowed recovery action when appropriate, and verify whether the incident has actually been resolved. Thus, LogRCA contributes to the evidence-analysis foundation of the proposed system, while the project combines that capability with agentic investigation, CI/CD context, safe remediation and closed-loop verification.

4. A Comprehensive Survey on Root Cause Analysis in (Micro) Services: Methodologies, Challenges, and Trends

Author(s): Tingting Wang; Guilin Qi

DOI / Identifier: arXiv:2408.00803v1 (the source contains a DOI placeholder, not a valid DOI)

Date: 23 July 2024

Review / Notes:

This survey provides a broad overview of Root Cause Analysis in microservice environments and is highly relevant to the proposed Autonomous CI/CD Incident Triager & Self-Healer because it establishes the different types of operational evidence that can be used to investigate incidents. It organizes RCA approaches around metrics, traces, logs, multi-modal data and the emerging role of Large Language Models. This is directly related to the proposed project's objective of allowing an AI agent to collect and correlate evidence from multiple sources instead of relying on a single alert or log message.

The survey describes RCA as a process that begins with data collection, followed by analysis of the collected evidence and then formulation and validation of possible root causes. This structure closely matches the proposed project methodology. When an incident is received, our system should first gather relevant operational and CI/CD information, analyze the evidence to identify patterns and relationships, and then generate and evaluate possible root-cause hypotheses.

The survey's discussion of metrics is relevant to the monitoring component of the proposed system. Metrics such as CPU utilization, memory consumption, network activity, request rates and latency can indicate which service or component is affected. For our project, these metrics can be used to determine whether an incident is related to resource exhaustion or service degradation and can also be used after remediation to verify whether the system has recovered.

The discussion of traces is relevant to distributed-service investigation. In a microservice architecture, a user request may pass through several services, making it difficult to determine where a failure originated. Trace information can help the proposed agent follow the path of an affected request and relate failures between dependent services.

The survey's treatment of logs is also important for our project. Logs provide detailed information about application and infrastructure events, but their large volume and variability make direct analysis difficult. The proposed incident agent can therefore retrieve the relevant log window, identify important error events and combine those events with metrics, traces and other operational information before making a root-cause decision.

A particularly important part of the survey for the proposed project is its discussion of multi-modal RCA. The survey emphasizes that metrics, traces and logs provide different views of the same incident. Combining them can provide stronger evidence than relying on any single source. Our proposed system extends this multi-source idea beyond runtime observability by also considering Git changes, CI/CD pipeline results, deployment history and configuration changes. This allows the agent to connect an operational failure with events in the software-delivery lifecycle.

The survey also discusses the role of LLMs in RCA. This supports the proposed use of an LLM as the reasoning and orchestration component of the incident agent. Rather than expecting the LLM to infer a root cause from a fixed prompt, the proposed system can use it to decide what evidence should be collected, call controlled diagnostic tools, compare evidence and generate explainable root-cause hypotheses.

The survey identifies several challenges that are directly relevant to the proposed project, including complex dependencies between services, propagation of failures across a system, noisy and heterogeneous operational

data, and the difficulty of accurately localizing the original fault. These challenges justify the project's emphasis on timeline construction, evidence correlation and hypothesis-based investigation.

Overall, this survey provides the conceptual foundation for the RCA portion of the proposed Autonomous CI/CD Incident Triager & Self-Healer. It supports a methodology in which the incident agent combines multiple evidence sources, reasons about relationships between events and services, identifies likely root causes and uses the resulting diagnosis as the basis for a controlled recovery action. The proposed project places this RCA capability inside a larger CI/CD incident-response workflow that continues through remediation and post-action verification.

5. Root Cause Analysis for Microservices based on Causal Inference: How Far Are We?

Author(s): Luan Pham; Huong Ha; Hongyu Zhang

DOI / Identifier: 10.1145/3691620.3695065

Date: 8 September 2024

Review / Notes:

This paper studies Root Cause Analysis for microservices from the perspective of causal inference. Its relevance to the proposed Autonomous CI/CD Incident Triager & Self-Healer comes from addressing a central problem in incident investigation: the component that shows an abnormal symptom is not necessarily the component that caused the incident. In a distributed microservice system, failures can propagate through service dependencies, so identifying a correlation between two abnormal events does not automatically establish which event caused the other.

For the proposed project, this distinction is important because the incident agent is expected to identify the most likely underlying cause rather than simply report the service that generated the largest number of errors. For example, a deployment may introduce an incorrect configuration in one service, while the visible symptoms appear later as database errors, increased latency or failures in dependent services. A causal perspective encourages the investigation to consider the dependency structure and the direction of the failure rather than treating every correlated signal as an equally likely root cause.

The paper is therefore relevant to the proposed system's root-cause-analysis module and its use of evidence from distributed services. The agent can combine operational observations with information about service relationships, recent changes and incident timing to determine which component or change is more likely to have initiated the failure. This complements the proposed project's timeline-based investigation, where events such as a Git commit, build, deployment, configuration change and subsequent service degradation are ordered to help distinguish possible causes from downstream effects.

The paper also highlights that causal RCA methods have different assumptions and strengths and that no single causal approach can be expected to perform equally well for every microservice failure scenario. This is important for our project because the proposed agent should not rely on one fixed RCA algorithm as the only source of truth. Instead, causal reasoning can be one part of an evidence-based investigation in which the agent also considers logs, metrics, traces, deployment history, Git changes and CI/CD information.

This connects with the project's planned hypothesis-based reasoning. The agent can generate multiple possible explanations for an incident and use additional evidence to determine which explanation is most consistent with the observed sequence of events and system dependencies. Causal information can strengthen this process by helping the agent distinguish a probable initiating fault from symptoms that appeared later in the dependency chain.

The paper is also relevant to the evaluation strategy of the proposed project. It demonstrates the importance of testing RCA approaches across different failure conditions rather than assuming that good performance on one type of incident will generalize to all incidents. For our controlled prototype, this supports the use of multiple incident scenarios, including deployment regressions, configuration errors, service failures and dependency-related failures.

Within the scope of the proposed Autonomous CI/CD Incident Triager & Self-Healer, the main value of this work is therefore the causal perspective on RCA. It supports designing the incident agent to reason about dependencies, event ordering and possible cause-and-effect relationships instead of relying only on anomaly correlation. The proposed project combines this perspective with multi-source evidence collection and CI/CD context, and then uses the resulting diagnosis to select a controlled remediation action and verify whether the incident has been resolved.

6. RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data

Author(s): Luan Pham; Hongyu Zhang; Huong Ha; Flora Salim; Xiuzhen Zhang

DOI / Identifier: 10.1145/3701716.3715290

Date: 3 February 2025

Review / Notes:

Pham et al. (2025) introduce RCAEval as an open-source benchmark designed to address a major methodological problem in root cause analysis (RCA) for microservice systems: the lack of large-scale, reproducible datasets and comprehensive evaluation environments. The authors observe that existing RCA studies are often evaluated on only a small number of systems and fault types, making it difficult to compare techniques fairly or determine how well they generalize to realistic failures. RCAEval therefore shifts attention from proposing another isolated RCA algorithm to establishing a common experimental foundation on which RCA methods can be developed, compared, and reproduced.

The benchmark contains 735 failure cases collected from three microservice systems—Google Online Boutique, Weaveworks Sock Shop, and Train Ticket—and covers 11 fault types. Importantly, the benchmark combines multiple telemetry modalities: metrics, logs, and distributed traces. The datasets are organized into RE1, RE2, and RE3. RE1 contains metric-based failures, while RE2 extends the setting to multi-source RCA using metrics, logs, and traces. RE3 focuses on code-level faults and similarly provides multi-source telemetry. Each failure case is annotated with the root-cause service and a root-cause indicator, such as a specific metric or log, which supports both coarse-grained localization and more fine-grained analysis. The benchmark consequently provides a substantially richer basis for evaluating RCA than approaches that depend only on a single telemetry source or synthetic data.

The paper is particularly relevant to the proposed Autonomous CI/CD Incident Triager & Self-Healer because the project also treats incident investigation as a multi-source evidence-correlation problem. The project specification identifies application and infrastructure logs, CI/CD pipeline logs, monitoring metrics, deployment history, Git commits, pull requests, configuration changes, and service-health information as potential evidence sources. RCAEval establishes the importance of combining heterogeneous operational evidence rather than depending on one signal. Its use of metrics, logs, and traces provides a strong foundation for the project's telemetry-analysis layer, while the project's additional emphasis on Git, deployment, and CI/CD history extends the incident context toward the software-delivery lifecycle.

RCAEval also reinforces the importance of distinguishing symptoms from root causes. In distributed microservice systems, a fault in one component can propagate through service dependencies and create symptoms in several downstream services. Consequently, the most visible anomaly is not necessarily the initiating fault. This is closely aligned with the proposed agent's investigation workflow, where the system is expected to construct a timeline, generate multiple root-cause hypotheses, and evaluate supporting and contradicting evidence instead of immediately selecting the first observed error. RCAEval's annotated root-cause indicators provide an appropriate reference for evaluating whether an autonomous investigator can identify the underlying cause rather than merely detect abnormal services.

Another important contribution of RCAEval is its reproducible evaluation framework. The paper provides fifteen baselines spanning metric-based, trace-based, and multi-source RCA and supports evaluation at both coarse-grained and fine-grained levels. This is important for the proposed project because an autonomous incident agent should be evaluated systematically rather than only through qualitative demonstrations. RCAEval suggests a model for controlled experiments in which known faults are injected, telemetry is collected, and the predicted root cause is compared with ground truth. The project can adopt this evaluation philosophy while extending the measured outcome from RCA alone to the complete incident-response workflow.

The benchmark's scale also highlights the practical difficulty of RCA. RE2 contains very large quantities of telemetry, including millions of log lines and millions of traces for the benchmark systems. This supports an important design principle for the proposed agent: it should not blindly pass all available operational data to an LLM. Instead, it should selectively retrieve and summarize evidence relevant to the incident. This connects directly with the project's Incident Context Builder and Data Collector, where the agent is intended to determine what information is missing, query relevant data sources, and construct a structured incident timeline. The same evidence-reduction principle is visible in other supplied literature: LogRCA seeks a minimal set of causally relevant log lines, while MicroRCA-Agent applies filtering and summarization before multimodal reasoning. RCAEval's inclusion of code-level faults is especially relevant to a CI/CD-oriented incident triager. The RE3 dataset evaluates faults introduced into application source code and uses telemetry such as logs and traces to diagnose them. This demonstrates that production symptoms can be connected back to defects at the code level rather than only to infrastructure anomalies. For the proposed project, this motivates correlating telemetry with recent Git commits, pull requests, deployments, and configuration changes. A CI/CD incident may therefore be interpreted as a chain from source change to build or deployment to runtime symptoms, rather than as an isolated monitoring event.

The broader supplied literature complements RCAEval by showing how different evidence types and reasoning mechanisms can be combined. Wang and Qi's survey organizes microservice RCA into metric-based, trace-based, log-based, multi-modal, and LLM-assisted approaches, emphasizing the need to analyze heterogeneous operational evidence. Roy et al. show that LLM agents can improve RCA by dynamically retrieving additional diagnostic information rather than relying only on an incident description. Flow-of-Action further demonstrates that unconstrained LLM agents can make irrelevant or hallucinated tool calls and therefore benefit from structured procedures, auxiliary agents, and controlled action selection. These findings are directly applicable to the proposed system's design: evidence collection should be tool-driven, reasoning should be explicit, and autonomous actions should be constrained by safety policies. For the proposed Autonomous CI/CD Incident Triager & Self-Healer, RCAEval is therefore valuable not primarily as a self-healing system, but as a benchmark and evaluation foundation for the root-cause-analysis component. It provides realistic fault diversity, multi-source telemetry, annotated causes, and reproducible baselines. The project can use these ideas to construct controlled incident scenarios involving bad deployments, configuration errors, application crashes, resource exhaustion, dependency problems, and database connection failures. The agent can then be evaluated on root-cause accuracy and evidence quality before adding the recovery-planning and closed-loop self-healing stages.

Overall, RCAEval establishes that reliable RCA requires broad and reproducible evaluation across heterogeneous microservice environments and telemetry sources. Its benchmark-oriented contribution complements the proposed project's agentic approach: RCAEval provides the experimental foundation for asking whether an autonomous agent can correctly identify a root cause, while the proposed system extends the workflow toward dynamic evidence collection, CI/CD and source-code correlation, recovery planning, safety-aware execution, and post-remediation verification. Thus, the paper is highly relevant to the project's RCA module and, more broadly, to its research methodology and evaluation strategy.

7. AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds

Author(s): Yinfang Chen; Manish Shetty; Gagan Somashekar; Minghua Ma; Yogesh Simmhan; Jonathan Mace; Chetan Bansal; Rujia Wang; Saravan Rajmohan

DOI / Identifier: arXiv:2501.06706v1

Date: 12 January 2025

Review / Notes:

Chen et al. (2025) present AIOpsLab, a holistic framework for designing, developing, and evaluating AI agents for autonomous cloud operations. The paper is motivated by the increasing complexity of distributed cloud and microservice environments and by the limited ability of existing AIOps benchmarks to evaluate AI agents across a complete incident lifecycle. Rather than evaluating an agent only on a static dataset or a single operational task, AIOpsLab creates an interactive environment in which an agent can observe a running cloud system, investigate a fault, take actions, receive feedback, and be evaluated against a predefined expected outcome. The authors describe this broader operational-agent paradigm as AgentOps, where AI agents can manage multiple cross-layer operations rather than performing isolated detection or classification tasks.

A central contribution of AIOpsLab is its unified evaluation framework. The system combines a microservice cloud environment, workload generation, fault injection, telemetry collection, an orchestrator, and an Agent-Cloud Interface (ACI). The ACI provides the controlled interaction boundary through which an agent communicates with the cloud, takes actions, and receives feedback. The framework therefore models incident response as an interactive process rather than as a one-time prediction. This is highly relevant to the proposed Autonomous CI/CD Incident Triager & Self-Healer because the proposed project follows the same general investigation loop: an incident is received, relevant evidence is collected, hypotheses are formed and tested, a recovery action is planned or executed, and the resulting system state is verified.

AIOpsLab formalizes an operational problem as P = <T, C, S>, where T represents the task, C represents the context, and S represents the expected solution or oracle. The task taxonomy contains four levels: detection, localization, root-cause analysis, and mitigation. Detection determines whether a fault is present; localization identifies the affected service or component; RCA identifies the system layer and fault type; and mitigation requires the agent to interact with the environment to correct the fault, for example by updating configuration or rolling back to a previous version. This progression maps closely to the proposed project's workflow from incident detection and triage through root-cause analysis and controlled remediation. It also supports the idea that incident handling should be evaluated as a sequence of increasingly difficult operational capabilities rather than as a single RCA score.

The framework uses two microservice applications from DeathStarBench as testbeds and supports fault injection through ChaosMesh. Its observability layer uses Prometheus for metrics, Jaeger for distributed tracing, and Filebeat and Logstash for logging. Kubernetes and Helm APIs are integrated into the orchestrator. This multi-source telemetry design is directly relevant to the proposed project because our incident investigator is expected to reason over logs, metrics, service health, deployment information, and other operational evidence. AIOpsLab demonstrates how an agent can interact with telemetry in a live environment instead of receiving a fixed, pre-collected incident record.

Another important contribution is the benchmark itself. The paper constructs 48 evaluation problems covering different AIOps tasks and evaluates multiple LLM-based agents. The problems are designed around realistic incidents and fine-grained root causes rather than only simple performance degradation or application crashes. The benchmark measures correctness as well as operational efficiency, including Time-to-Detect (TTD), Time-to-Mitigate (TTM), number of agent actions or steps, and token usage. This evaluation approach is especially relevant to the proposed project because our intended evaluation should measure not only whether the agent

identifies the correct root cause, but also how quickly and efficiently it investigates, whether its remediation succeeds, and how many actions are required.

The experimental results highlight an important characteristic of autonomous incident management: detection and localization are easier than complete RCA and mitigation. The paper reports that the RCA and mitigation tasks are the most challenging for the evaluated agents, and that none of the agents consistently achieves high performance across all four task categories. The authors also show that increasing the number of agent steps can improve performance for some agents, but that performance may eventually plateau. In the reported experiments, FLASH reaches its highest accuracy of 59.32% with a step limit of 20, while increasing the step limit for GPT-3.5-Turbo beyond a certain point mainly increases token consumption without improving accuracy. These findings support the proposed project's emphasis on purposeful investigation rather than unrestricted agent exploration.

AIOpsLab also identifies failure modes that are important for the proposed system. Agents can struggle with complex operational tasks, use unnecessary actions, misinterpret normal system behaviour as an incident, and have difficulty completing mitigation. The paper reports false-positive detection behaviour on no-fault scenarios and notes that some agents may produce explanations that are inconsistent with the actual fault or workload. The use of an Agent-Cloud Interface as a guard-rail and the use of predefined evaluators are therefore important design ideas. These ideas align with the proposed project's safety layer, where the AI should not receive unrestricted infrastructure access and remediation actions should pass through policies, allowlists, approval gates, dry-run execution, and post-action verification.

The paper is also important from an evaluation perspective because it argues that static telemetry datasets and fixed question-answer benchmarks do not adequately represent dynamic cloud operations. A real incident changes the state of the environment as investigation actions are performed, and a mitigation action can itself change the subsequent evidence available to the agent. AIOpsLab addresses this by allowing agents to interact with an environment and by evaluating the final system state. This is closely aligned with the proposed project's closed-loop self-healing objective: after a recovery action, the agent should inspect error rates, service health, logs, and other indicators to determine whether the incident has actually been resolved. If the system has not recovered, the agent should continue investigating rather than incorrectly declaring success.

The supplied literature complements AIOpsLab in several ways. RCAEval provides a reproducible benchmark for microservice RCA using telemetry and annotated failure cases, while AIOpsLab extends the evaluation setting into an interactive environment where agents can act on the system. LogRCA emphasizes reducing large volumes of noisy log data to a minimal set of root-cause-relevant evidence. The LLM-agent RCA work by Roy et al. demonstrates the value of dynamically retrieving additional diagnostic information through tools. Flow-of-Action further shows that unconstrained LLM agents can suffer from hallucinations and irrelevant actions and therefore benefit from SOP-guided, multi-agent coordination. Together, these works support the proposed architecture in which evidence collection, reasoning, action selection, safety controls, and verification are treated as connected stages.

For the proposed Autonomous CI/CD Incident Triager & Self-Healer, AIOpsLab is particularly useful as a methodological reference for building and evaluating the prototype. The project's initial MVP can follow a similar controlled-environment philosophy: create known incident scenarios, expose the agent to operational evidence, allow it to use predefined investigation tools, evaluate its root-cause hypothesis against ground truth, and then measure the result of a controlled remediation. However, the proposed project places additional emphasis on the CI/CD lifecycle by explicitly incorporating Git changes, deployment history, CI/CD pipeline information, configuration changes, and the relationship between a software change and the subsequent production incident.

Overall, AIOpsLab demonstrates that meaningful evaluation of autonomous AIOps agents requires more than a static dataset or an LLM response judged in isolation. Agents need realistic operational environments, dynamic telemetry, controlled fault scenarios, interactive tool access, explicit success criteria, and measurements covering both correctness and operational efficiency. These principles directly support the proposed project's

methodology and evaluation plan, particularly its multi-source evidence collection, agentic investigation loop, controlled remediation, and post-remediation verification.

8. Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis

Author(s): Changhua Pei; Zexin Wang; Fengrui Liu; Zeyan Li; Yang Liu; Xiao He; Rong Kang; Tieying Zhang; Jianjun Chen; Jianhui Li; Gaogang Xie; Dan Pei

DOI / Identifier: 10.1145/3701716.3715225

Date: 12 February 2025

Review / Notes:

Pei et al. (2025) propose Flow-of-Action, a Standard Operating Procedure (SOP)-enhanced LLM-based multi-agent system for root cause analysis in microservice environments. The paper addresses an important limitation of using general-purpose LLM agents for RCA: although the ReAct thought-action-observation paradigm resembles the way Site Reliability Engineers investigate incidents, an LLM can hallucinate, select irrelevant tools, or become overwhelmed by complex and variable observations. These errors can influence subsequent investigation steps and lead the agent away from the actual root cause. Flow-of-Action therefore focuses on constraining and structuring the investigation process rather than allowing the LLM to select every action freely.

The central idea is to encode experienced SRE diagnostic procedures as Standard Operating Procedures and use them as guidance during RCA. The authors introduce an SOP-centric framework called SOP flow. It contains tools for finding the SOPs relevant to an incident, automatically generating an SOP when a suitable procedure is unavailable, and converting SOPs into executable code. The SOP is used as a soft constraint: it guides the main agent toward an appropriate diagnostic trajectory while retaining flexibility when unexpected circumstances occur. This is important for the proposed Autonomous CI/CD Incident Triager & Self-Healer because the project also requires an agent to investigate incidents dynamically, but within a controlled and explainable workflow rather than through unrestricted LLM reasoning.

Flow-of-Action also changes the action-selection process. Instead of immediately choosing one tool or action after every observation, the system introduces a thought-actionset-action-observation paradigm. The agent first reasons about a set of plausible actions and then selects an appropriate action. This is intended to reduce inefficient trial-and-error tool invocation when multiple actions could reasonably follow from the same observation. For the proposed project, this principle can be applied to the investigation stage: after receiving an incident, the agent can identify the information it needs—such as logs, deployment history, Git changes, metrics, or service health—and select the most relevant next investigation step rather than blindly calling every available tool.

The paper further divides responsibilities among multiple agents. The MainAgent coordinates the RCA process, while CodeAgent assists with code-related investigation, JudgeAgent helps determine whether the root cause has been identified, ObAgent extracts important information and fault types from large amounts of observational data, and ActionAgent assists with generating and selecting action sets. This separation of responsibilities is directly relevant to the proposed project's agentic architecture. Our system similarly benefits from specialized tools and modules for log retrieval, Git analysis, CI/CD analysis, deployment history, root-cause reasoning, recovery planning, and verification. Flow-of-Action demonstrates that distributing complex RCA responsibilities can reduce the cognitive burden on a single LLM and improve the overall investigation process.

A major contribution of the paper is its treatment of information overload. RCA environments can expose agents to large amounts of logs, metrics, traces, code information, and other observations. ObAgent is designed to extract fault types and key information from this data before it is used by the main agent. This is closely connected to the proposed project's Incident Context Builder and Data Collector. The project specification explicitly proposes collecting logs, metrics, CI/CD information, Git changes, deployments and configuration changes, but the agent should not simply provide all raw data to the LLM. Instead, evidence should be filtered, structured, correlated into a timeline, and presented in a form that supports root-cause reasoning. The paper's approach is also relevant to explainability and repeatability. SOPs represent explicit diagnostic

knowledge: they describe why particular investigation steps should be performed and provide a structured path through an incident. In the proposed project, a similar concept can be used for remediation and investigation policies. For example, a deployment-related incident may trigger a procedure that checks the deployment timeline, recent commits, configuration changes, application errors, and service health before recommending a rollback. This provides a more auditable process than allowing the LLM to invent an investigation procedure from scratch.

The experimental results demonstrate the value of these constraints. The paper reports that the ReAct baseline achieves 35.50% RCA accuracy, while Flow-of-Action achieves 64.01%. The authors attribute the improvement to the combination of SOP-guided orchestration, the action-set mechanism, and multi-agent collaboration. Ablation results further indicate that removing individual agents reduces accuracy to some degree, while removing the action-set mechanism has a particularly noticeable effect on action efficiency. The paper also limits the number of investigation steps to prevent unbounded loops. These findings are highly relevant to the proposed system because they show that autonomous RCA should be designed around controlled reasoning, bounded tool usage, and explicit stopping criteria rather than unlimited agent exploration.

Flow-of-Action also complements the other supplied literature. RCAEval provides a benchmark for evaluating RCA using metrics, logs, and traces across multiple microservice systems, while AIOpsLab evaluates agents in an interactive cloud environment where they can detect, localize, diagnose, and mitigate faults. LogRCA emphasizes selecting useful log evidence from very large log collections, and the LLM-agent RCA work demonstrates the importance of allowing an agent to dynamically retrieve diagnostic information. Flow-of-Action contributes the additional idea that this tool use should be guided by structured operational knowledge and coordinated among specialized agents. Together, these works support the proposed project's architecture of evidence collection, structured reasoning, controlled action selection, and verification.

For the Autonomous CI/CD Incident Triager & Self-Healer, Flow-of-Action is especially relevant to the investigation and decision-making layers. The proposed system should be able to receive an incident, determine which evidence is required, call appropriate investigation tools, generate multiple hypotheses, rank them using evidence, and select a recovery action. Flow-of-Action suggests that this process should be guided by predefined operational procedures and should use bounded, reasoned action selection. The proposed project can therefore incorporate SOP-like investigation policies for recurring incident classes such as bad deployments, configuration errors, failed pipelines, application crashes, dependency problems, and database connection failures.

The paper is also relevant to the project's safety model. Flow-of-Action demonstrates that unrestricted LLM tool selection can result in irrelevant actions and inefficient investigation. In the proposed self-healing system, the same principle should be extended beyond investigation to remediation: the LLM should not execute arbitrary commands. Instead, it should propose an action through a constrained action catalogue and policy layer, with human approval required for high-risk operations. The final remediation should then be followed by health checks and telemetry-based verification.

Overall, Flow-of-Action provides strong support for an agentic RCA architecture in which LLM reasoning is combined with structured operational knowledge, specialized agents, evidence filtering, bounded action selection, and explicit stopping conditions. Its findings are directly applicable to the proposed Autonomous CI/CD Incident Triager & Self-Healer because the project aims to move from simple LLM-generated incident explanations toward an autonomous investigation loop. The paper's SOP-guided approach provides a practical mechanism for making that loop more reliable, explainable, and safer while still retaining the flexibility of LLM-based reasoning.

9. Cloud-Based AI Systems: Leveraging Large Language Models for Intelligent Fault Detection and Autonomous Self-Healing

Author(s): Cheng Ji; Huaiying Luo

DOI / Identifier: arXiv:2505.11743v1 **Date:** 16 May 2025

Review / Notes:

Ji and Luo (2025) propose a cloud-based AI framework that combines Large Language Models (LLMs) with machine-learning-based fault detection to support intelligent fault detection and autonomous self-healing in cloud systems. The paper is motivated by the increasing scale and complexity of cloud infrastructure, where traditional rule-based detection and human intervention can become difficult to maintain. The proposed framework uses the semantic understanding capabilities of an LLM to process system logs, error reports, and real-time data streams, while conventional machine-learning techniques are used for fault classification and anomaly detection. The overall objective is to move cloud operations from reactive fault handling toward earlier detection, diagnosis, and automated recovery.

The framework follows a multi-level architecture. It combines supervised learning for fault classification with unsupervised learning for anomaly detection, allowing the system to identify abnormal behaviour and classify potential fault conditions. The LLM component provides semantic interpretation of operational information such as logs and error messages, while the machine-learning components provide structured detection capabilities. Once a fault is identified, the framework can automatically trigger a self-healing mechanism. This combination is relevant to the proposed Autonomous CI/CD Incident Triager & Self-Healer because our system also needs to combine automated operational-data analysis with higher-level reasoning rather than relying exclusively on fixed rules.

A particularly relevant aspect of the paper is its treatment of unstructured operational information. Cloud incidents generate logs, error messages, reports, and other textual signals that are difficult to process effectively using traditional numerical anomaly-detection techniques alone. The paper uses LLM natural-language understanding to interpret this information semantically. For the proposed project, this supports the use of an LLM as the reasoning layer over heterogeneous operational evidence. Application logs and CI/CD logs can contain different descriptions of the same underlying incident, while Git commits, deployment changes, configuration changes, and alert messages may provide additional context. An LLM can help transform these heterogeneous observations into a structured incident context that can then be used for root-cause reasoning.

The paper also compares several fault-detection approaches, including Transformer-based fault detection, Graph Neural Network (GNN)-based fault detection, and Deep Reinforcement Learning (DRL) for autonomous self-healing. This is significant for the proposed project because it demonstrates that fault management is not a single-model problem. Different techniques can be useful for different stages: anomaly-detection models can identify abnormal behaviour, graph-based approaches can represent relationships in distributed systems, reinforcement learning can support recovery decisions, and LLMs can interpret unstructured operational information. The proposed project similarly separates incident detection, evidence collection, root-cause analysis, recovery planning, and verification instead of attempting to make one model perform every operational task.

The self-healing component of the paper is particularly relevant to the second half of the proposed project. Self-healing systems are intended to automatically respond to failures through corrective actions such as restarting services, reallocating resources, or applying corrective changes. Ji and Luo argue that detection alone is insufficient because a practical cloud reliability system must also reduce downtime through automated recovery. Their framework therefore links fault detection to an automatic self-healing mechanism. This corresponds to the proposed project's Recovery Planner and Safe Action modules, where a likely root cause is converted into a controlled recovery action such as rollback, restart, scaling, or retry.

However, the proposed project places more emphasis on the reasoning process between detection and remediation. Instead of moving directly from an anomaly to a recovery action, our agent is intended to construct an incident timeline, collect additional evidence, generate multiple root-cause hypotheses, rank those hypotheses using supporting and contradicting evidence, and only then select an appropriate recovery action. The Ji and Luo framework therefore provides a useful foundation for the detection-to-self-healing relationship, while the proposed project extends the workflow toward explicit agentic investigation and evidence-based RCA.

The paper's use of multiple operational data sources is also closely related to the proposed system's observability layer. Our project is designed to collect application and infrastructure logs, CI/CD pipeline logs, monitoring metrics, deployment history, Git changes, configuration changes, and service-health information. Ji and Luo specifically emphasize logs, error reports, and real-time data streams as inputs to the LLM-based framework. This supports the project's decision to treat operational information as a heterogeneous evidence space rather than relying on a single monitoring signal.

The paper is complementary to the other supplied RCA literature. RCAEval demonstrates the importance of evaluating RCA across multiple telemetry sources and realistic microservice fault scenarios. LogRCA focuses on identifying the small amount of root-cause-relevant information within large volumes of logs. Roy et al. demonstrate that LLM agents can dynamically retrieve additional diagnostic information during RCA. AIOpsLab evaluates agents in interactive cloud environments and shows that complete RCA and mitigation are more difficult than simple detection or localization. Flow-of-Action further demonstrates that LLM-based investigation benefits from structured procedures and constrained action selection. Together, these studies suggest that the proposed system should combine detection, evidence retrieval, reasoning, controlled action selection, and verification rather than treating self-healing as a simple automatic response to an alert.

Ji and Luo also report improvements over traditional fault-detection approaches in fault-detection accuracy, downtime reduction, and recovery speed. These outcomes are directly related to the evaluation goals of the proposed project. Our system should not be evaluated only on whether it can generate a plausible root-cause explanation. It should also be evaluated on operational outcomes such as root-cause accuracy, investigation time, remediation success rate, false-remediation rate, recovery time, and whether the system returns to a healthy state after the selected action.

For the Autonomous CI/CD Incident Triager & Self-Healer, the paper is especially useful in establishing the connection between intelligent fault detection and autonomous recovery. It supports the idea that LLMs can add semantic understanding to operational data while machine-learning components can provide structured anomaly detection and classification. At the same time, the proposed project focuses more specifically on the CI/CD incident lifecycle by incorporating recent source-code changes, pipeline results, deployment history, and configuration changes into the investigation process.

Overall, Ji and Luo provide a relevant foundation for combining LLM-based semantic interpretation, machine-learning fault detection, and autonomous self-healing in cloud environments. The work supports the proposed project's use of AI for incident detection and recovery, while the project's agentic architecture builds a more explicit investigation loop around these capabilities: detect the incident, dynamically collect relevant evidence, construct a timeline, generate and rank root-cause hypotheses, plan a safe remediation, execute only permitted actions, and verify the post-remediation state. This makes the paper directly relevant to both the project's Incident Detection/RCA components and its eventual self-healing workflow.

10. MicroRCA-Agent: Microservice Root Cause Analysis Method Based on Large Language Model Agents

Author(s): Pan Tang; Shixiang Tang; Huanqi Pu; Zhiqing Miao; Zhixing Wang

DOI / Identifier: arXiv:2509.15635v1

Date: 19 September 2025

Review / Notes:

Overall, MicroRCA-Agent demonstrates that effective LLM-based RCA in microservice environments depends not only on the reasoning capability of the LLM, but also on the quality, compression, alignment, and integration of the evidence provided to it. Its modular multimodal architecture provides a strong reference for the proposed project's evidence-driven RCA layer. The proposed Autonomous CI/CD Incident Triager & Self-Healer builds on this direction by adding CI/CD and source-change context, agentic evidence retrieval, hypothesis-based reasoning, constrained remediation, and closed-loop verification, thereby connecting multimodal RCA with the broader incident-response and self-healing workflow.

The paper also supports the project's evaluation strategy. MicroRCA-Agent demonstrates the value of ablation experiments for understanding which evidence sources contribute to RCA. A similar approach can be used in the proposed project by comparing an agent with logs only, logs plus metrics, logs plus metrics plus traces, and finally the complete CI/CD-aware evidence set including Git and deployment information. Alongside root-cause accuracy, the proposed project can evaluate investigation time, evidence quality, number of investigation steps, remediation success, false remediation, and recovery time.

For the proposed Autonomous CI/CD Incident Triager & Self-Healer, MicroRCA-Agent is particularly relevant to the Evidence Collection, Incident Context Builder, and RCA Engine modules. The project can adopt its principle of modality-specific preprocessing: logs can be parsed and filtered, metrics can be reduced to relevant anomalies, and traces or service-health information can be summarized before being provided to the reasoning agent. The project then extends this operational context with CI/CD-specific evidence such as recent Git commits, pull requests, deployment history, pipeline results, and configuration changes. This allows the agent to reason not only about where a failure is occurring, but also about what changed immediately before it.

The paper complements several other supplied studies. RCAEval establishes a reproducible benchmark for microservice RCA using metrics, logs, and traces, while AIOpsLab evaluates AI agents in interactive cloud environments covering detection, localization, RCA, and mitigation. LogRCA focuses on extracting causally relevant information from large log volumes, and Roy et al. show that LLM agents benefit from dynamically retrieving additional diagnostic information. Flow-of-Action adds SOP-guided and multi-agent control to reduce irrelevant or hallucinated agent actions. MicroRCA-Agent fits into this research progression by showing how carefully preprocessed multimodal telemetry can be combined with LLM reasoning for practical microservice RCA.

The paper's results also show the importance of managing LLM context and hallucination risk. MicroRCA-Agent uses filtering, summarization, structured prompts, and modular processing to control the information supplied to the LLM. The reported analysis includes cases where LLM reasoning can still produce hallucinations, demonstrating that multimodal LLM-based RCA is not automatically reliable simply because multiple data sources are provided. This is relevant to the proposed system's safety architecture. The project should constrain the LLM through predefined tools, structured evidence, action allowlists, policy checks, and post-action verification rather than allowing unrestricted infrastructure actions.

MicroRCA-Agent also provides an important lesson about the role of LLMs in operational systems. The LLM is not used as a replacement for all conventional monitoring and anomaly-detection methods. Instead, traditional algorithms are used to reduce and structure the operational data, while the LLM is used where semantic interpretation and cross-modal reasoning are valuable. This hybrid design is directly relevant to the proposed

project. Our system can use conventional observability tools and deterministic queries for data retrieval and basic anomaly detection, while the agent performs hypothesis generation, evidence correlation, root-cause ranking, and recovery planning. This division can reduce unnecessary LLM calls and make the system more explainable and controllable.

A particularly important finding is the complementary value of the different modalities. The paper's ablation analysis shows that removing or isolating modalities changes performance and that combining modalities can improve RCA quality. In particular, the reported results highlight strong synergy between log and metric information, while the metric component also makes a substantial individual contribution. This supports a central design decision in the proposed project: incident diagnosis should not rely only on application logs or only on monitoring metrics. Logs can provide semantic descriptions of failures, metrics can identify quantitative deviations, and traces can show distributed request behaviour. Combining these signals provides a more complete incident context.

The Multimodal Root Cause Analysis module is the core decision-making stage. It integrates the processed log features, anomalous trace information, and summarized metric phenomena and uses carefully designed cross-modal prompts to reason over the combined evidence. The output is structured around the faulty component, root-cause description, and reasoning trace. This directly corresponds to the proposed project's requirement for evidence-based RCA rather than a simple natural-language answer. Our proposed agent should similarly produce the most likely root cause together with supporting evidence, confidence, and an explanation of why alternative hypotheses are less likely.
The Metric Fault Summarization module is another important component. Rather than sending all metric dimensions to the LLM, the system first uses a statistical symmetric-ratio filtering strategy to identify potentially abnormal metrics and remove normal or redundant information. It then uses a two-stage LLM analysis to summarize phenomena hierarchically across service, pod, and node levels. This is especially relevant to the proposed project's Incident Context Builder. Our system should similarly transform raw monitoring information into a structured description of what changed, where the change occurred, and how it relates to the incident. The hierarchical analysis also fits the distributed nature of cloud-native systems, where an incident may simultaneously produce symptoms at the service, container/pod, and infrastructure levels.

The Trace Anomaly Detection module combines Isolation Forest with status-code validation. This gives the system two complementary signals for identifying anomalous request traces: statistical anomaly detection for abnormal durations and validation of request status behaviour. The approach is useful for the proposed project because traces can provide information that logs alone cannot: they can show how a request propagates across distributed services and where abnormal behaviour occurs in the request path. In a CI/CD incident, this can help distinguish an application-level regression from an infrastructure-wide problem and can provide additional evidence for ranking root-cause hypotheses.

The Log Fault Extraction module uses the Drain log-parsing algorithm together with multi-level filtering to reduce massive quantities of raw log data to structured fault features. The filtering process uses the fault time window, error-related keywords, relevant fields, fault templates, deduplication, frequency statistics, and service-name mapping. The resulting information can include the affected node, service, pod, error message, and frequency. This is highly relevant to the proposed project because a production incident can generate a large volume of repetitive logs, and passing all of them directly to an LLM would be inefficient. The project therefore benefits from the same principle: retrieve and compress relevant evidence before asking the reasoning model to identify the root cause.

The Data Preprocessing module standardizes the input data and aligns timestamps across logs, traces, and metrics. This is important because the three telemetry sources can represent the same incident using different time formats and granularities. Accurate temporal alignment allows observations from different modalities to be correlated around the same fault window. This idea directly supports the proposed project's timeline-based reasoning. Our agent is intended to correlate a sequence such as Git change → build → deployment → metric degradation → application errors → incident alert. MicroRCA-Agent shows the value of first establishing a consistent temporal context before attempting multimodal reasoning.

The architecture is organized into five main modules: Data Preprocessing, Log Fault Extraction, Trace Anomaly Detection, Metric Fault Summarization, and Multimodal Root Cause Analysis. This modular design is particularly relevant to the proposed Autonomous CI/CD Incident Triager & Self-Healer because our system also separates data collection, evidence construction, RCA, recovery planning, and verification into distinct functional components. MicroRCA-Agent demonstrates that multimodal RCA can benefit from specialized processing for each data type before the final reasoning stage, rather than forcing one general-purpose model to perform raw-data processing and causal reasoning simultaneously.

Tang et al. (2025) propose MicroRCA-Agent, an LLM-agent-based method for root cause analysis in microservice systems that combines logs, distributed traces, and metrics through a modular multimodal analysis pipeline. The paper addresses a central problem in modern microservice operations: operational data is heterogeneous, high-volume, and difficult to correlate, while directly providing raw monitoring data to an LLM can exceed its context capacity and increase the possibility of unreliable reasoning. Instead of treating the LLM as a direct consumer of all raw telemetry, MicroRCA-Agent first preprocesses and compresses each modality and then provides structured fault information to an LLM-based multimodal RCA module. The paper reports a final score of 50.71 in the 2025 CCF International AIOps Challenge and uses ablation experiments to examine the contribution of different data modalities.

11. Multi-Agent Systems for Root Cause Analysis in Microservices

Author(s): Alexander Naakka; Yuqing Wang; Mika V. Mäntylä

DOI / Identifier: arXiv:2605.03505v1

Date: 5 May 2026

Review / Notes:

Naakka, Wang, and Mäntylä (2026) propose LATS-RCA, an LLM-based multi-agent framework for root cause analysis in microservice systems. The work addresses the difficulty of manual RCA in distributed microservice environments, where engineers must correlate diverse operational signals and reason about several potentially responsible services under time pressure. The authors identify a limitation in earlier LLM-based RCA approaches: they generally follow a linear reasoning path, which can be inadequate when an observed anomaly initially implicates multiple services or components. LATS-RCA addresses this by formulating RCA as a reflection-guided tree search in which multiple candidate root-cause hypotheses can be explored, evaluated, and compared rather than committing to one diagnostic path too early.

The core of LATS-RCA is the Language Agent Tree Search mechanism, combining LLM reflection with Monte Carlo Tree Search. Multiple LLM-driven agents iteratively reason over execution logs and performance metrics, while the search maintains competing hypotheses and expands promising diagnostic paths. Reflection evaluates intermediate reasoning states using evidence quality, diagnostic completeness, and internal consistency. This is highly relevant to the proposed Autonomous CI/CD Incident Triager & Self-Healer because our project also intends to generate several root-cause hypotheses and rank them using supporting and contradicting evidence rather than selecting the first plausible explanation.

The architecture contains specialized log and metric agents coordinated by a supervisor. Each agent investigates its own observability modality, while the supervisor controls sequencing, evaluates progress, performs cross-modal correlation, and can hand off a condensed evidence summary when additional analysis is required. This directly relates to the proposed project's agentic architecture, where different capabilities handle logs, metrics, Git, CI/CD, deployment history, RCA, recovery planning, and verification. The paper demonstrates the value of specialization while retaining centralized coordination.

A particularly important contribution is the explicit maintenance of multiple candidate hypotheses. Rather than treating RCA as a single chain of reasoning, LATS-RCA explores alternative explanations and uses reflection to decide which paths deserve further investigation. This aligns with the proposed RCA Engine: an increase in HTTP 500 errors could initially correspond to a bad deployment, database failure, configuration error, or resource exhaustion. The agent can collect additional evidence and update hypothesis confidence before recommending remediation.

On the LO2 benchmark, LATS-RCA achieves 91.3% diagnostic accuracy, compared with 39.8% for single-agent ReAct and 57.4% for multi-agent ReAct. The ablation study shows that candidate batching, backpropagation, and reflection all contribute to performance, with removal of candidate batching reducing accuracy to 84.3%. These results support structured hypothesis exploration over simple linear reasoning.
The accuracy improvement comes with substantially higher computational cost. LATS-RCA averages 53.1 API calls, 156K tokens, and 9.1 minutes per investigation, compared with 13.5 calls, 41K tokens, and 1.2 minutes for single-agent ReAct. It explores an average of 18.9 hypotheses per investigation. This is important for our project because an incident-response agent cannot simply maximize reasoning depth; investigation needs bounded tool-call, token, time, and stopping budgets.

The production validation is especially relevant. LATS-RCA was deployed in a polyglot production microservice environment with Node.js, Rust, Python, and Go services and evaluated on 37 real-world incidents. Accuracy was approximately 60–70%, lower than the 91.3% achieved on LO2. The production

environment contained heterogeneous logs, timestamps, severity conventions, metric names, and coverage, demonstrating that real-world operational heterogeneity substantially affects agentic RCA.
This supports a key design principle for the proposed project: evidence must be normalized and correlated before reasoning. Logs, metrics, Git changes, deployment events, and CI/CD results should be transformed into a consistent incident context and timeline. The proposed system can extend this idea beyond observability by explicitly connecting source-code changes and deployments to subsequent production symptoms.

The paper complements RCAEval, AIOpsLab, Flow-of-Action, MicroRCA-Agent, and other supplied literature. RCAEval provides reproducible multi-source RCA evaluation; AIOpsLab evaluates agents across detection, localization, RCA, and mitigation; Flow-of-Action constrains LLM agents using SOPs and structured action selection; and MicroRCA-Agent demonstrates modality-specific preprocessing before multimodal LLM reasoning. Together, these studies support an architecture based on evidence collection, specialized reasoning, controlled actions, and verification.

For the Autonomous CI/CD Incident Triager & Self-Healer, LATS-RCA is especially relevant to the AI Incident Analyzer and RCA Engine. The project can adopt competing-hypothesis exploration while extending the evidence space to Git commits, pull requests, CI/CD results, deployment history, configuration changes, and service health. This allows the agent to reason not only about where an incident is occurring, but also about what changed immediately before it.

The paper also informs the project's evaluation strategy. In addition to root-cause accuracy, the project should measure investigation time, tool/API calls, tokens, number of hypotheses explored, remediation success, false remediation, and recovery time. This is important because LATS-RCA demonstrates that higher diagnostic accuracy can require substantially greater computational cost.

Overall, Multi-Agent Systems for Root Cause Analysis in Microservices demonstrates that structured multi-agent exploration of competing hypotheses can substantially improve RCA compared with linear reasoning, while also showing the practical cost and production-heterogeneity challenges of such systems. It provides a strong foundation for the proposed project's hypothesis-generation and evidence-driven reasoning layer, while our project connects that reasoning to CI/CD-aware investigation, constrained remediation, and closed-loop verification.

12. GALA: Graph-Augmented LLM Agents for Root Cause Analysis and Incident Response in Microservices

Author(s): Yifang Tian; Yaming Liu; Zichun Chong; Zihang Huang; Yiran Li; Hans-Arno Jacobsen

DOI / Identifier: 10.1145/3832783.3834335

Date: 10 August 2026 (arXiv v1 date; ASE ’26 publication)

Review / Notes:

Tian et al. (2026) present GALA: Graph-Augmented LLM Agents for Root Cause Analysis and Incident Response in Microservices. The paper addresses three closely related challenges in microservice RCA: the true root cause may be different from the service where the symptom is observed; different telemetry modalities may point to different candidate causes; and identifying the faulty service alone is insufficient because an SRE also needs a causal explanation and actionable response. GALA therefore combines service-dependency graphs, multiple telemetry modalities, statistical hypothesis generation, and LLM-based agentic investigation. This makes it highly relevant to the proposed Autonomous CI/CD Incident Triager & Self-Healer, which similarly aims to correlate heterogeneous operational evidence and move from detection to diagnosis and controlled recovery.

The framework treats metrics, logs, and distributed traces as complementary evidence sources. Metrics provide quantitative information such as CPU usage and latency, logs provide timestamped event information, and traces capture cross-service calls and timing relationships. The paper constructs a service dependency graph from distributed traces so that the agent can reason about the topology through which failures propagate. This is directly relevant to the proposed project because an incident may appear in one component while the actual fault originates elsewhere. In our system, the same principle can be extended beyond runtime service dependencies to include relationships between CI/CD pipelines, deployments, Git changes, configuration changes, and affected services.

GALA+ uses a four-phase architecture. Phase I performs initial root-cause hypothesis generation using complementary metric- and trace-based signals. Phase II creates pod-centric diagnostic bundles containing relevant telemetry while preserving temporal and dependency context. Phase III performs graph-guided agentic investigation, and Phase IV consolidates the investigation into a final diagnosis, incident summary, and prioritized action recommendations. This staged design is closely aligned with the proposed project's flow of incident detection, data collection, AI incident analysis, RCA, recovery planning, safe action, and post-incident reporting.

A major contribution is the use of dual-signal hypothesis generation. GALA+ uses BARO for coarse metric-based ranking and introduces STRIX for trace- and graph-structure-aware ranking. The two rankings are fused into a consolidated candidate list before LLM agents perform deeper investigation. The paper deliberately does not use logs in the initial ranking because some failures can be silent and therefore produce weak or no error logs; logs are instead incorporated during later diagnostic investigation. This is a useful design principle for our project: no single evidence source should automatically dominate the investigation, and evidence should be used at the stage where it is most informative.

The pod-centric diagnostic bundle is another important component. Raw telemetry is distilled into a compact representation that preserves temporal patterns, dependency context, and error signals. This reduces the amount of information that must be processed by the LLM and allows the agent to focus on evidence relevant to each candidate service or pod. This directly supports our proposed Incident Context Builder, which needs to collect and compress logs, metrics, CI/CD information, deployment history, Git changes, and configuration changes before passing them to the reasoning layer.

The graph-guided investigation mechanism is particularly relevant to the proposed project's agentic reasoning. GALA+ launches investigation branches for the highest-ranked candidates in parallel. Each Investigator Agent

evaluates a candidate using its diagnostic bundle and produces a structured assessment containing confidence, evidence summary, causal explanation, and supporting and contradicting factors. If confidence is insufficient, the agent expands the investigation through dependency neighbors in bounded batches. This creates a controlled search process: the agent does not inspect every service indiscriminately, but follows plausible fault-propagation paths and stops when sufficient evidence has been gathered.

This approach closely matches the proposed project's requirement to generate multiple root-cause hypotheses and rank them using evidence. For example, a production incident following a deployment might have several plausible causes: a defective code change, a configuration change, a database problem, resource exhaustion, or a downstream dependency failure. Rather than immediately selecting the most obvious anomaly, the proposed agent can maintain several hypotheses, collect evidence for each, and use dependency and temporal relationships to decide which explanation is strongest.

GALA+ also demonstrates why graph-guided exploration can be preferable to unrestricted LLM tool use. The paper reports that the earlier GALA workflow could gather insufficient evidence because it investigated one retrieved telemetry bundle at a time, while unrestricted ReAct exploration could drift away from plausible root-cause candidates. GALA+ addresses this by constraining exploration using the service dependency graph and confidence-based, depth-bounded expansion. The graph therefore functions as a reasoning constraint rather than merely as a retrieval structure.

The results show strong performance. On the Online Boutique and Train Ticket benchmarks, GALA+ achieves 74.44% and 73.33% AC@1 respectively and outperforms the strongest LLM-based baseline by more than 25 percentage points. The paper also evaluates AegisLab, which contains 25 fault types, where GALA+ achieves the best reported MRR, AC@1, and AC@3 among the evaluated methods. These results support the value of combining graph structure, multimodal evidence, and agentic investigation rather than relying on a single telemetry signal or unconstrained LLM reasoning.

The paper does not evaluate only root-cause ranking. It also evaluates incident summarization and action recommendations using SURE-Score, a human-guided evaluation framework developed with SRE expertise. SURE-Score assesses dimensions related to the usefulness and quality of incident reports, and human SRE evaluation confirms GALA+'s advantage over Flow-of-Action across the evaluated dimensions. This is particularly relevant to our project because the intended output is not just a root-cause label. The system should produce an explainable incident report containing the evidence, causal reasoning, recommended recovery, and operationally useful next steps.

GALA+ also provides useful evidence about the cost of agentic reasoning. The system requires approximately one minute per incident on average and consumes about 32.7K tokens on Online Boutique and 90.4K on Train Ticket. The authors explicitly treat this as a trade-off: graph-guided pruning reduces unnecessary exploration while retaining stronger diagnostic performance. The parameter study shows that too few investigation branches can reduce coverage, while too many introduce noisy hypotheses; a moderate pruning threshold provides a better balance between investigation depth and efficiency. This is directly relevant to our project because an autonomous incident investigator must balance diagnostic accuracy against response time, API/tool calls, token usage, and operational cost.

The paper's comparison with other supplied literature is also important. RCAEval establishes reproducible multi-source RCA evaluation, AIOpsLab evaluates interactive agents across detection, localization, RCA, and mitigation, Flow-of-Action constrains agent behaviour using SOPs and structured action selection, and MicroRCA-Agent demonstrates modality-specific preprocessing before multimodal LLM reasoning. GALA+ combines several of these ideas by using modality-specific initial ranking, compact evidence bundles, graph-constrained agentic investigation, and structured incident-response generation. This makes it a particularly relevant recent reference for the proposed project's architecture.

For the Autonomous CI/CD Incident Triager & Self-Healer, GALA+ is most relevant to the Incident Context Builder, RCA Engine, and Recovery Planner. Its graph-guided reasoning can inspire a broader evidence graph in

which nodes represent services, deployments, commits, pipeline executions, configuration changes, alerts, and infrastructure resources, while edges represent relationships such as deployed-by, changed-before, depends-on, triggered, or affected-by. Such a graph could help the agent focus investigation on the most plausible causal paths instead of searching the entire operational environment.

The incident-response output of GALA+ is also directly aligned with the proposed project's goal of controlled remediation. The paper generates stratified action recommendations covering immediate mitigation, permanent fix, and preventative measures. Our project can extend this concept by connecting recommendations to a safety layer: low-risk actions can be automatically executed from an allowlisted action set, while high-risk actions can require human approval. After execution, the system should verify service health and incident indicators before declaring recovery.

Finally, GALA+ highlights several practical considerations that are important for the proposed system. The authors note that the approach depends on access to all three telemetry modalities, that the evaluation uses controlled fault-injection benchmarks, and that token cost grows with candidate services and exploration coverage. They also note that adapting the system to different deployments requires additional prompt engineering. These observations reinforce the need for our prototype to have a controlled scope, standardized incident scenarios, clear tool interfaces, bounded agent actions, and explicit evaluation metrics.

Overall, GALA+ provides a strong recent foundation for the proposed project's evidence-driven RCA architecture. It demonstrates how heterogeneous telemetry can be combined with service topology and LLM agents to investigate multiple hypotheses, suppress false positives, generate causal explanations, and produce actionable incident-response recommendations. The proposed Autonomous CI/CD Incident Triager & Self-Healer can build on these ideas while expanding the evidence graph to include the software-delivery lifecycle Git, CI/CD pipelines, deployments, and configuration changes and connecting graph-guided RCA to constrained remediation and closed-loop post-recovery verification.

13. LLM for Automated Root Cause Analysis in Microservices Architectures

Author(s): Amruta Mhatre; Afrid Shaikh

Date: Accepted: 30 January 2025

Review / Notes:

Mhatre and Shaikh (2025) propose an automated Root Cause Analysis (RCA) system for microservices architectures using Large Language Models combined with a machine-learning-based exception classifier. The paper focuses on the large volume of business and runtime exceptions generated by microservice applications and the limitations of manual exception analysis. Its main contribution is a layered architecture that aggregates exceptions, classifies them, and then uses fine-tuned LLMs to perform code-aware RCA from runtime stack traces and relevant source-code context. The approach is particularly relevant to the proposed Autonomous CI/CD Incident Triager & Self-Healer because it demonstrates how LLMs can connect runtime failures with the underlying application code rather than treating an incident only as a monitoring anomaly.

The system begins with an exception aggregation layer built around Apache Kafka. Exceptions generated by multiple microservices are collected into a Kafka topic and forwarded to an Exception Classifier. This classifier preprocesses error messages and categorizes exceptions into business exceptions and runtime exceptions. Runtime exceptions are then sent to the RCA service. This separation is useful because not every exception requires the same diagnostic treatment, and filtering the event stream before invoking the LLM can reduce unnecessary reasoning and improve efficiency. For the proposed project, a similar staged approach can be used to distinguish informational events, expected application errors, deployment failures, and incidents that require deeper autonomous investigation.

The Exception Classifier compares traditional machine-learning approaches including XGBoost, Support Vector Machine, and Naïve Bayes with LLM-based classification. The paper reports that XGBoost performs best among the selected conventional classification models. This is an important design lesson for the proposed system: an LLM does not necessarily need to replace every conventional component. Deterministic rules or lightweight machine-learning models can perform high-volume filtering and classification, while the LLM can be reserved for tasks requiring contextual interpretation and reasoning. This hybrid architecture is consistent with the broader project design, where operational tools collect evidence and the AI agent performs higher-level correlation and root-cause reasoning.

The RCA service is based on fine-tuned CodeLlama and Mistral models. It receives runtime exception stack traces together with tokenized or preprocessed code-repository information and generates a root-cause analysis. The paper reports that CodeLlama-7B performs particularly well when supplied with preprocessed code snippets. The approach therefore establishes an important connection between runtime symptoms and source-code context. For the proposed CI/CD incident triager, this is directly relevant because recent Git commits and pull requests can be used as evidence when an incident occurs after a deployment. Instead of asking only “which service failed?”, the agent can investigate “what changed in the code or configuration immediately before the failure?”

The paper's layered architecture also has a natural correspondence with the proposed project's workflow. Kafka-based exception aggregation corresponds to the project's Incident Detection and Data Collection layer; exception classification corresponds to initial incident triage; the LLM-based RCA service corresponds to the RCA Engine; and the repository-aware analysis provides a foundation for correlating runtime symptoms with software changes. The proposed project extends this workflow beyond exception-driven diagnosis by incorporating monitoring metrics, deployment history, CI/CD pipeline results, configuration changes, and service-health information.
A major strength of the work is its code-aware diagnostic approach. Stack traces can identify where an exception occurred, but understanding the actual cause may require examining the relevant source code and the relationships between functions or components. By providing preprocessed code snippets to the LLM, the system attempts to move from exception classification toward causal explanation. This is highly relevant to the

proposed project because production incidents can be triggered by an incorrect code change even when the most visible symptom is a generic HTTP error, timeout, or service failure.

The paper reports more than 90% precision for business-exception classification, 89.6% recall for runtime-exception classification, less than one second for RCA diagnostics per exception, and more than 85% accuracy in human qualitative evaluation. These results demonstrate the potential of combining automated classification with code-aware LLM reasoning for rapid diagnosis. For the proposed project, the emphasis on speed is particularly relevant because incident response is time-sensitive. However, our evaluation needs to extend beyond diagnosis latency to include multi-source RCA accuracy, investigation steps, remediation success, recovery time, and verification reliability.

The paper also identifies several directions that connect directly with the proposed system. The authors recognize that their current RCA relies primarily on exception stack traces and code context and suggest incorporating broader environment logs such as resource consumption and disk/network usage. They also propose feedback loops for adapting the model as services evolve and identify autonomous remediation as a future extension. These observations closely match the direction of our project: the proposed agent is intended to combine application and infrastructure logs, metrics, Git changes, deployment history, CI/CD information, and configuration changes, and then continue from diagnosis to safe remediation and post-remediation verification.

The paper is complementary to the other supplied literature. RCAEval demonstrates the value of multi-source telemetry and reproducible RCA benchmarks. MicroRCA-Agent combines logs, traces, and metrics through modality-specific preprocessing before multimodal LLM reasoning. GALA uses service dependency graphs and graph-guided investigation to reason about fault propagation. Flow-of-Action introduces SOP-guided multi-agent coordination to control LLM tool use. AIOpsLab evaluates agents in interactive cloud environments across detection, localization, RCA, and mitigation. Together, these studies show a progression from specialized fault analysis toward autonomous, evidence-driven incident investigation. The Mhatre and Shaikh work adds an important software-level perspective by explicitly connecting runtime exceptions with source-code context.

For the Autonomous CI/CD Incident Triager & Self-Healer, this paper is particularly useful for the Code and Exception Analysis part of the RCA Engine. The project can use a similar principle in which a runtime failure triggers targeted retrieval of relevant source-code changes, recent commits, pull requests, and deployment information. This evidence can then be correlated with logs and metrics to determine whether a recent software change is a plausible root cause. The system should avoid sending the entire repository to the LLM; instead, it should retrieve only the files, functions, commits, or configuration changes that are relevant to the observed failure.

The paper also supports the project's use of event aggregation before agentic reasoning. A production environment may generate many repeated exceptions from multiple instances or services. Aggregating, deduplicating, classifying, and prioritizing these events can prevent the reasoning agent from wasting resources on duplicate symptoms. The proposed Incident Context Builder can extend this concept by grouping alerts and events into a single incident timeline and associating them with deployments and code changes.

Finally, the paper provides a useful bridge between RCA and remediation. Its authors explicitly identify autonomous remediation and broader environmental context as future directions. In the proposed project, remediation is treated as a controlled stage after RCA: the agent generates a recovery plan, checks safety policies, optionally executes an allowlisted action, and verifies whether the system returns to a healthy state. Thus, the paper provides a strong foundation for the code-aware diagnosis portion of the proposed system while the project extends the workflow toward multi-source evidence correlation, CI/CD-aware investigation, constrained self-healing, and closed-loop verification.

Overall, Mhatre and Shaikh demonstrate that LLMs can perform useful automated RCA when runtime exceptions are combined with relevant source-code context, and that a hybrid architecture can efficiently separate high-volume classification from deeper semantic diagnosis. The work is directly relevant to the proposed Autonomous CI/CD Incident Triager & Self-Healer because it supports connecting production

symptoms to recent software changes. Its limitations and stated future directions further motivate the proposed project's broader evidence model, which integrates logs, metrics, Git, CI/CD pipelines, deployments, and configuration changes before selecting and verifying a recovery action.

14. AI-Augmented Self-Healing Infrastructure: Combining Health Probes with Remediation Playbooks

Author(s): Manoj Kumar Reddy Kalakoti

Date: Accepted: 18 August 2025

Review / Notes:

Kalakoti (2025) presents an AI-augmented self-healing infrastructure framework that combines continuous health monitoring, AI-assisted incident diagnosis, and predefined remediation playbooks. The work focuses on a practical problem in modern cloud and distributed infrastructure: monitoring systems can detect that a service is unhealthy, but detection alone does not guarantee fast or reliable recovery. The proposed approach therefore connects health probes with AI-driven interpretation and executable remediation playbooks, creating a feedback loop in which an operational problem can be detected, diagnosed, remediated, and checked again. This is directly relevant to the proposed Autonomous CI/CD Incident Triager & Self-Healer because the project also aims to move beyond alert generation toward controlled autonomous recovery.

The framework uses Kubernetes as the infrastructure environment and combines health probes, monitoring data, an AI reasoning layer, and remediation actions. Kubernetes liveness, readiness, and startup probes provide continuous signals about application health and availability. When an unhealthy condition is detected, the system can invoke AI-assisted reasoning and select a suitable remediation playbook. The use of health probes as the trigger mechanism is important for the proposed project because the system needs a reliable way to determine whether an incident is still active and whether a remediation action has actually restored service health.

A key contribution is the combination of AI reasoning with predefined remediation playbooks. Instead of allowing an LLM to invent arbitrary infrastructure commands, the system uses structured playbooks that represent known recovery procedures. This provides a controlled action space and improves operational safety. For the proposed project, this principle is particularly important because self-healing actions can have significant consequences. The Recovery Planner should therefore generate or select a recovery plan, but execution should occur through an allowlisted set of safe actions such as restarting a service, rolling back a deployment, scaling a workload, restarting a failed component, or re-running a controlled pipeline step.

The paper's architecture can be viewed as a closed-loop sequence: health monitoring identifies an abnormal condition, AI-assisted analysis helps determine the likely cause, a remediation playbook is selected, the action is executed, and health is monitored again to determine whether the issue has been resolved. This closely matches the proposed project's intended workflow of incident detection, evidence collection, root-cause analysis, recovery planning, safe action execution, and post-remediation verification. The main difference in emphasis is that the proposed project places stronger focus on the investigation stage and on correlating operational symptoms with CI/CD and source-code changes.

The use of health probes also demonstrates the importance of objective verification. An LLM-generated statement that an incident has been fixed is not sufficient evidence of recovery. The system needs measurable signals such as service availability, error rate, latency, pod health, deployment status, and application logs. For the proposed project, this suggests that the Post-Remediation Verification module should independently check the system state after every recovery action. If health does not improve, the agent should treat the remediation as unsuccessful and return to investigation rather than terminating the incident automatically.

The framework is also relevant to observability. Health probes provide a simple but important layer of service-state information, while monitoring systems provide richer operational signals. In the proposed project, this concept can be expanded to include application and infrastructure logs, metrics, CI/CD pipeline logs, deployment history, Git commits, pull requests, configuration changes, and service-health information. These

sources can be combined into a structured incident timeline before the AI reasoning layer selects a remediation playbook.

The paper emphasizes the practical value of automation in reducing manual intervention and improving recovery consistency. Predefined remediation playbooks can encode operational knowledge and make common recovery actions repeatable. This complements the SOP-oriented approach in Flow-of-Action, where structured procedures constrain LLM investigation, and it complements the multi-agent approaches in AIOpsLab, LATS-RCA, and GALA that use AI to perform more complex diagnosis. Together, these studies suggest a layered architecture in which AI provides reasoning and decision support while deterministic operational procedures provide controlled execution.

The relationship with RCA research is particularly important for the proposed project. Health-probe-based self-healing can answer "is the service unhealthy?" and playbooks can answer "what known recovery action should be attempted?", but reliable autonomous incident handling also requires answering "why did the service become unhealthy?" RCAEval, MicroRCA-Agent, GALA, and other supplied papers address this diagnostic problem through telemetry correlation, multimodal reasoning, graphs, and LLM agents. The proposed project combines these capabilities by placing evidence-driven RCA before playbook selection, so that remediation is tied to a specific root-cause hypothesis rather than being triggered only by a generic health failure.

The paper also supports the idea of using different remediation levels. Low-risk actions can be executed automatically, while actions with greater potential impact can require approval. This is important for the proposed project's Safety Layer. For example, restarting an unhealthy pod may be considered a low-risk automated action, while rolling back a production deployment or changing infrastructure configuration may require stronger validation or human approval. This policy-driven execution model reduces the risk of an LLM making an unsafe operational decision.

For the Autonomous CI/CD Incident Triager & Self-Healer, the paper is particularly relevant to the Recovery Planner, Safe Action Executor, and Post-Remediation Verification modules. The project can adopt the concept of remediation playbooks and map each approved root-cause category to a set of permitted recovery actions. For example, a failed deployment may lead to a rollback playbook, a transient dependency failure may trigger a controlled retry, a resource-exhaustion incident may trigger scaling, and a crashed service may trigger a restart. The agent should select from these actions based on evidence rather than generating unrestricted commands.

The paper also suggests an important mechanism for learning and improvement: operational outcomes can be fed back into the incident-management process. If a playbook repeatedly succeeds for a particular failure pattern, it can become a more strongly trusted recovery path. If a remediation repeatedly fails, the system can record the failure and require deeper investigation or human review. The proposed project can use incident reports, remediation outcomes, and verification results to build a history of what actions worked for particular incident classes.

The broader supplied literature strengthens this design. AIOpsLab demonstrates that autonomous agents should be evaluated in interactive environments where mitigation changes system state. Flow-of-Action demonstrates the need for constrained action selection. RCAEval provides reproducible fault scenarios and ground truth for RCA evaluation. GALA introduces graph-guided investigation and actionable response generation. MicroRCA-Agent demonstrates the importance of preprocessing heterogeneous telemetry before LLM reasoning. The self-healing framework in this paper provides the complementary execution and verification layer that connects diagnosis to recovery.
For the proposed project, this combination suggests that self-healing should not be implemented as a single LLM command. Instead, the system should follow a controlled pipeline: detect the incident, collect and normalize evidence, generate and rank root-cause hypotheses, choose a remediation playbook, check safety policy and approval requirements, execute the permitted action, and independently verify whether the system has recovered. This architecture preserves the advantages of AI-assisted reasoning while limiting the risks associated with unrestricted autonomous infrastructure access.

Overall, Kalakoti's work provides a strong practical foundation for the self-healing portion of the proposed Autonomous CI/CD Incident Triager & Self-Healer. Its combination of continuous health probes, AI-assisted diagnosis, remediation playbooks, and post-action monitoring demonstrates how automated recovery can be made more structured and repeatable. The proposed project builds on this idea by connecting playbook-driven recovery to multi-source, CI/CD-aware root-cause analysis and by adding explicit evidence collection, source-change correlation, safety controls, and closed-loop verification.

15. A Two-Staged LLM-Based Framework for CI/CD Failure Detection and Remediation with Industrial Validation

Author(s): Weiyuan Xu, Juntao Luo, Tao Huang, Kaixin Sui, Jie Geng, Qijun Ma, Isami Akasaka, Xiaoxue Shi, Jing Tang, Peng Cai

DOI / Identifier: 10.1109/ASE63991.2025.00310

Date: June 2025

Review / Notes:

This paper presents LogSage, an end-to-end Large Language Model-based framework designed specifically for detecting, diagnosing, and remediating failures in Continuous Integration and Continuous Deployment pipelines. The work is highly relevant to the proposed Autonomous CI/CD Incident Triager & Self-Healer because it directly addresses the problem of automated failure investigation within CI/CD environments. Unlike general-purpose incident management approaches, LogSage focuses on the unique characteristics of CI/CD pipeline logs, which are often lengthy, heterogeneous, noisy, and difficult to analyze manually.

The proposed LogSage framework follows a two-stage architecture. The first stage performs root cause analysis on failed CI/CD pipeline logs. Instead of directly providing entire raw logs to an LLM, LogSage introduces a specialized preprocessing pipeline that identifies important error information and removes irrelevant log content. This reduces the amount of information that must be processed by the language model and allows the reasoning component to focus on high-value diagnostic evidence. The output of this stage includes a structured diagnosis and a human-readable explanation of the identified failure.

The second stage focuses on solution generation and remediation. LogSage uses Retrieval-Augmented Generation to retrieve relevant historical resolution strategies from available knowledge sources. The retrieved information is combined with the RCA result so that the system can generate an appropriate solution for the detected failure. Tool-calling is then used to translate the generated solution into an actionable remediation procedure. This is particularly important for the proposed project because it demonstrates that an LLM-based system can progress beyond simply explaining an incident and can instead generate actions intended to resolve it.

A major contribution of this work is its evaluation in both a public dataset and a large-scale industrial environment. The RCA stage achieved approximately 98% precision with near-perfect recall, demonstrating that specialized preprocessing can substantially improve LLM-based analysis of CI/CD logs. The framework was also deployed in an industrial CI/CD environment where it processed more than 1.07 million executions over a year, demonstrating that LLM-based CI/CD failure management can operate at significant scale. The reported end-to-end precision exceeded 88%, showing the practical potential of combining automated diagnosis with solution generation.
For the proposed Autonomous CI/CD Incident Triager & Self-Healer, this paper provides strong support for the CI/CD failure analysis module. The project can adopt a similar principle of preprocessing and filtering logs before sending them to the reasoning model. Instead of treating the entire CI/CD log as context, the proposed system can extract critical errors, identify relevant stages of the pipeline, construct an incident timeline, and combine the resulting evidence with deployment and source-code information.

However, the proposed project can extend the scope of LogSage beyond CI/CD pipeline logs. LogSage primarily focuses on failures within the CI/CD environment, whereas the proposed system is intended to connect software delivery events with production incidents. For example, the proposed agent can investigate whether a failed deployment, recent Git commit, configuration modification, or dependency update is responsible for a production failure. It can correlate CI/CD logs with application logs, infrastructure metrics, deployment history, and service health information.

Another important extension is hypothesis-based root cause analysis. Rather than immediately accepting the first diagnosis generated by the LLM, the proposed system can generate multiple candidate root causes and evaluate each using evidence collected from different sources. For example, after detecting a database connection failure, the agent could consider database outage, incorrect configuration, network failure, or a faulty deployment as separate hypotheses. It could then collect additional evidence to determine which explanation is most strongly supported.

The paper also provides an important foundation for the proposed Recovery Planner. LogSage demonstrates how retrieved historical knowledge and tool calling can be used to produce actionable remediation. The proposed system can extend this idea by introducing a dedicated safety layer between the LLM and infrastructure. Actions such as deployment rollback, service restart, pipeline retry, and controlled scaling can be placed on an allowlist, while potentially destructive operations can require human approval.

The proposed system can further extend the work through closed-loop verification. Generating a remediation action does not necessarily mean that the incident has been resolved. After executing a rollback or restart, the agent should monitor error rates, service health, logs, and other relevant metrics. If the system returns to a healthy state, the incident can be marked as resolved. If the failure persists, the agent should return to the investigation phase and reconsider its hypotheses.

Therefore, the main research opportunity derived from this paper is to extend automated CI/CD failure diagnosis into a broader evidence-driven incident-response lifecycle. The proposed project combines CI/CD logs and historical resolution knowledge with Git changes, deployment information, production telemetry, hypothesis-based reasoning, controlled remediation, and post-remediation verification. This creates a broader investigation-to-recovery workflow rather than a system limited to CI/CD failure diagnosis.

16. Integrating Large Language Models with Cloud-Native Observability for Automated Root Cause Analysis and Remediation

Author(s): Chen Wang, Tingzhou Yuan, Cancan Hua, Lu Chang, Xiao Yang, Zhimin Qiu

DOI / Identifier: 10.1145/3797161.3797213

Review / Notes:

This paper presents a framework that integrates Large Language Models with cloud-native observability systems to automate root cause analysis and remediation in distributed cloud environments. The work is closely related to the proposed Autonomous CI/CD Incident Triager & Self-Healer because it addresses the complete operational pipeline from telemetry collection and anomaly detection to root cause identification and automated recovery.

The paper identifies a major challenge in modern cloud-native systems: applications are increasingly composed of large numbers of microservices, containers, and distributed components. These systems generate substantial amounts of operational data in the form of metrics, logs, and distributed traces. Traditional rule-based monitoring systems can detect predefined conditions, but they have difficulty understanding complex relationships between different signals. Human operators must often correlate these sources manually before determining the actual cause of an incident.

To address this challenge, the proposed architecture consists of several major layers. The first layer performs telemetry collection using OpenTelemetry. It collects metrics, logs, and traces from cloud-native applications. Metrics provide information about system and application performance, logs provide detailed event information, and traces provide information about the path of requests through different services. Combining these sources allows the system to obtain a more complete representation of an incident.

The next layer performs feature extraction and anomaly detection. Traditional machine learning techniques are used to process the collected telemetry before it is provided to the LLM. The framework uses techniques such as time-series anomaly detection, semantic analysis of logs, and service-dependency information. This design is important because it prevents the LLM from having to process the entire raw observability dataset. Instead, potentially relevant anomalies and evidence are identified first and passed to the reasoning layer.

The central component of the framework is an LLM-enhanced root cause analysis layer. A domain-adapted LLM is used to reason over the collected operational information and generate root-cause hypotheses. The system is designed to provide explanations for its diagnosis rather than simply returning a fault label. This aligns closely with the proposed project requirement that every root-cause prediction should be supported by evidence such as logs, metrics, deployment events, or source-code changes.

The paper also introduces a domain adaptation strategy for the LLM. Operational data contains specialized terminology, system behaviors, and failure patterns that may not be adequately represented in general-purpose language-model training data. The proposed framework therefore uses operational data to adapt the model and improve its ability to reason about cloud-native incidents. This demonstrates the importance of domain-specific information when using LLMs for AIOps and incident management.

Another significant component is the automated remediation layer. Once the LLM identifies a probable root cause, the system generates a remediation plan and translates it into operational actions. Importantly, the architecture incorporates safety mechanisms around remediation rather than allowing unrestricted execution of arbitrary model-generated commands. This concept is directly applicable to the proposed project, where infrastructure actions should be restricted through policies, allowlists, approval mechanisms, and controlled execution.

The framework also incorporates a feedback loop between remediation and system monitoring. After an action is performed, the system can observe the resulting system state and use the outcome to improve the operational process. This is an important characteristic of a self-healing system because simply executing a recovery command does not guarantee that the original incident has been resolved. The system must verify whether the relevant symptoms have disappeared.

Experimental evaluation reported substantial improvements in incident-management performance. The proposed framework achieved approximately 95% F1-score for anomaly detection, an 84.2% reduction in mean time to resolution compared with rule-based approaches, and an automation rate of approximately 91% for common incidents. These results demonstrate the potential of combining traditional observability techniques with LLM-based reasoning and automated remediation.

For the proposed Autonomous CI/CD Incident Triager & Self-Healer, this paper provides the foundation for the production observability side of the architecture. The proposed system can similarly collect metrics, logs, and traces and use these signals as evidence during incident investigation. However, rather than stopping at runtime telemetry, the project can combine these operational signals with software-delivery information such as Git commits, CI/CD pipeline executions, deployment versions, configuration changes, and pull requests.

This creates an important distinction between the proposed system and a conventional cloud-native self-healing framework. The proposed agent is intended to reason across the entire software-delivery and operational lifecycle. For example, if a service begins producing errors immediately after a deployment, the agent should not only inspect runtime telemetry. It should also investigate what changed in the new release, compare it with the previous healthy version, examine CI/CD results, and determine whether the deployment is a plausible cause.

The paper also supports the proposed project's use of multimodal evidence. Logs, metrics, and traces provide complementary views of the same incident. The proposed project can extend this multimodal approach by introducing a fourth category of evidence: software-delivery context. Git history, deployment records, pipeline results, configuration changes, and release information can be correlated with runtime telemetry to build a complete incident timeline.

A further extension is the introduction of explicit risk-aware remediation. The proposed system should classify recovery actions according to their potential impact. Low-risk actions such as restarting a failed service or retrying a pipeline may be automatically executed under predefined limits, while high-risk actions such as database modifications or destructive infrastructure changes should require human approval. This allows the system to retain the benefits of autonomous remediation while reducing the risks associated with unrestricted LLM-generated actions.

Finally, the proposed project can strengthen the closed-loop architecture by explicitly evaluating whether the remediation actually resolved the incident. After performing an action, the agent can compare the system's pre-action and post-action state using metrics, logs, traces, and health checks. If the incident persists, the system should not falsely report success. Instead, it should continue investigating alternative root-cause hypotheses.
Therefore, this paper provides a strong foundation for the observability, RCA, remediation, and feedback components of the proposed project. The research opportunity lies in integrating these capabilities with CI/CD and software-change information, thereby creating an agent capable of reasoning across the entire path from code change → CI/CD execution → deployment → production anomaly → RCA → controlled remediation → verification.